ling-engine: An Exact, Bandwidth-Bound Inference Engine for One Model on One Unified-Memory Workstation

dgxcoder
dgxcoder@dreamference.ai

October 2026

Abstract

A coding agent that runs on a single workstation spends almost all of its time waiting for its model. On the NVIDIA DGX Spark (GB10, 128 GB of unified LPDDR5X behind a 273 GB/s bus), decoding a 27-billion-parameter model is bound by memory bandwidth: every generated token streams the 17.6 GB of weights once, which caps plain decoding near 15 tokens/s however good the kernels are. We describe *ling-engine*, a C++20/CUDA inference engine built for exactly one model, Qwen3.8-27B in the mixed NVFP4/FP8 checkpoint that the Mightling agent serves, and for exactly one workload, that agent's recorded sessions. The engine keeps two invariants that general-purpose servers do not: greedy speculative output is token for token identical to plain decoding, and the recurrent state after a speculative run is bit for bit the state of a plain run, which lets every optimization be accepted or rejected by an equality test rather than a quality benchmark. Four measured milestones took the engine from 33.7 to 49.6 tokens/s of single-stream decode on replayed agent sessions, against 47–48 tokens/s for the production server (SGLang with the same drafter) on the same machine; prefill runs 1.1–1.3× production's; two agents served at once decode at 34.3 tokens/s per stream against production's 33.9. We report what did not pay, among them a context-lookup draft branch and a third pipeline stage, and one methodological finding: unseeded replays of sampled sessions carry ±5% of acceptance noise, enough to hide or invent every gain reported here, so every comparison is now made on seeded replays with identical output.

1 Introduction

Mightling is a local, air-gapped coding agent for a single DGX Spark. Its agent, `ling`, is a branded build of a vendor command-line agent, and its model server is SGLang [35] running Qwen3.8-27B with the DFlash2 block drafter [3, 34]. Measured over 168 recorded agent sessions (11,431 model requests), 92% of a session's wall time is spent in the model; a request writes a median of 98 and a mean of 177 tokens, 71% of them tool-call arguments; 46% of what the agent writes is copied from its own context; and every new session re-reads a system prompt and tool schema of about 11–15K tokens that all sessions share. Engine speed therefore turns almost one for one into agent speed, and the relevant benchmark is a replay of those sessions, not a chat benchmark.

A general server spends part of each decode step on work that is not weight traffic. ling-engine is the opposite bet: a compiler and a runtime for one model and one checkpoint, where every kernel is specialized to a matrix shape, every decode step is a captured CUDA graph, and the output is provably the same as plain decoding. This paper reports its design, the discipline that keeps it exact, four milestones of measurements on the same machine as production, and what did not work.

Tensor group	Precision	GB per pass
FFN gate, up, down (64 layers)	NVFP4	9.63
DeltaNet in- and out-projections (48 layers)	FP8	5.54
Attention q, k, v, o (16 layers)	FP8	1.68
LM head	NVFP4	0.72
β/α projections, norms, conv	BF16	0.05
Target weights per pass		17.6
Drafter layers	NVFP4	1.55

Table 1: What one decode or verify pass streams. At 262 GB/s the target weights alone take 67 ms.

Contributions. (1) A single-model engine whose speculative decoding is exact under two checkable rules, row invariance and commit by replay (section 4). (2) Measurements on one machine of where a speculative step's time goes, before and after each change (section 5). (3) Negative results on the context-lookup draft branch, pipeline depth, draft-block width under batching and FP8 KV on this hardware (section 6). (4) The observation that replay benchmarks of sampled sessions must be seeded to be comparable at all (section 7).

2 Hardware and model

The bus is the budget. A read-only streaming kernel sustains 262–264 GB/s on the GB10, 96–97% of the 273 GB/s peak; four SMs are enough to reach 98% of that, and DRAM latency is about 400 ns. CPU memory traffic shares the same pins byte for byte: two CPU threads streaming their own buffers take 27 GB/s from the GPU's read stream. The engine's targets are stated against 262 GB/s, and anything co-resident with a model server, such as an indexer, is budgeted as bus traffic.

The model. Qwen3.8-27B is a dense hybrid of 64 layers in 16 repeats of three Gated DeltaNet [31, 32] blocks followed by one gated full-attention block, each with its own SwiGLU FFN (hidden 5,120, intermediate 17,408), a 248,320-token vocabulary and a 262,144-token native context. The checkpoint Mightling serves is mixed precision: the FFNs and the LM head are NVFP4 (one FP8 scale per 16 values: the NVFP4 format [21, 22]), every DeltaNet and attention projection is FP8. One pass over the target weights reads 17.6 GB (table 1); moving the FP8 projections to NVFP4 would save 18%, which the checkpoint's makers chose not to do and which the engine therefore does not do either. Per-sequence state is small by comparison: a BF16 KV cache of 64 KB per token over the 16 attention layers and a DeltaNet state of 151 MB in FP32 that does not grow with context.

The drafter. DFlash2 drafts a block of B tokens in one pass from the target's residual stream after five of its layers, and a candidate selector walks a 16×16 lattice per position to one chain, which the target verifies in a single pass of B rows. Production serves $B = 16$; the engine runs the drafter exactly as production's code does and reproduces its acceptance, 4.98 against 4.95 tokens per step on the replay.

3 Design

Two programs. `ling-compile` is an offline step that fixes the precision map, folds the norms, lays the weights out on disk in the order the kernels read them, and instantiates one fused kernel per matrix shape. `ling-serve` is a resident runtime that keeps every session's KV cache and DeltaNet state on the GPU and speaks the agent's API, so the agent cannot tell which server answers.

The rows path. Every pass of 32 rows or fewer (a decode step, a verify, a short prompt) goes through one family of kernels: `stream_gemm`, a tensor-core weight-streaming GEMM for 1–32 rows that converts NVFP4 and FP8 weights to FP16 in registers exactly, with FP32 accumulation and split-K chosen from the matrix shape; a narrow BF16 kernel for the DeltaNet β/α projections; and tensor-core attention over fixed 512-key ranges. Weights are retiled at load into the order the kernel reads them; the retiling alone gave 5–6% end to end.

Prefill. Prompts of more than 32 rows go through a separate GEMM on block-scaled tensor cores (`mma.sync kind::mxf4nvf4` for NVFP4, `kind::f8f6f4` for FP8) over the same tiled weights, with activations quantized exactly as production's server quantizes them. The engine's own kernel was kept over CUTLASS 4.5.1 because it is faster on one of the two FFN shapes and slower on the other, and one tiled copy of the weights serves both paths.

One graph per step. A speculative step has two data-dependent decisions, the lattice walk after the draft and the accept rule after the verify, and both already ended in a synchronization. The three GPU phases between them, about 200, 1,300 and 100 launches, are each captured once into a CUDA graph [20] and replayed every step; positions and tokens travel through device memory and pinned host buffers at fixed addresses, and the commit has one graph per accepted count. Programmatic dependent launch lets each kernel on the rows path start as its predecessor drains, under one rule: every kernel waits on the grid dependency before its first read of a predecessor's output and before its first write.

Sessions and batches of two. The runtime holds a fixed number of resident sessions, each with its KV caches, DeltaNet and conv state, an end-of-prompt snapshot and a 2,048-position ring for the drafter's context; a request binds to the session whose history its prompt extends, and prefix checkpoints at shared-prefix boundaries let a new session resume after the system message every session shares. Two sessions decode in one pass: their verify rows are concatenated ($V_1 + V_2 \leq 32$), the per-row kernels take the second sequence unchanged, and the stateful kernels (attention, DeltaNet recurrence, the drafter's cache) run once over a device-side descriptor of both sequences. A fresh long prompt arriving while another session decodes is interleaved in 256-token chunks, so the decoding stream's step period stays within about twice its normal step.

4 Exactness

Speculative decoding preserves the output distribution in theory [2, 14]; the engine makes it hold in the bits, so that every change can be judged by equality. Two design rules carry it.

Row invariance. Every kernel on the rows path computes a row bit for bit the same whatever the other rows in the pass are: the GEMM uses a fixed K-chunk order per warp, a fixed-order warp reduction and a split-K never chosen from the row count; attention runs over fixed 512-key ranges at

fixed positions, where a tile past a row's last key leaves the online-softmax state exactly unchanged. A verify of 16 rows therefore computes for row i exactly what plain decoding would compute at that position.

Commit by replay. A verify reads the DeltaNet state and does not write it; it keeps each layer's projected inputs. The commit replays the accepted rows through the same convolution and recurrence kernels, whose per-token arithmetic does not depend on how many tokens one call covers. The state after a speculative run is thus the state of a plain run over the same tokens.

The gates. Every build runs `-spec-check` (each prompt decoded plainly and speculatively, tokens compared, then the speculative tokens replayed through plain decoding and every DeltaNet and conv state compared bit for bit), `-prefix-check` (a prompt resumed from a checkpoint reaches the state a prefill from scratch does), `-graph-check` (each prompt with and without graphs, greedy and seeded, requiring identical tokens, state and per-step acceptance), and from M4 on `-interleave-check` and `-batch-check` (each sequence of a pair equal to itself alone, in both row orders, with either one prefilling beside the other). Sampling is exact rejection sampling against the selector's 16-way draft distribution, tested by chi-square on a toy model. An optimization merges only when every gate passes, the output is the same as the previous main's, and it is no slower at one stream and at two. The graph check found one real defect: the first draft after every prompt had been running the drafter's projections through the prefill path.

5 Results

All measurements are on one development GB10 with production's SGLang image and flags as the reference, on a replay of recorded agent sessions (M0's main set: 12 sessions × 12 requests, production sampling, single stream unless stated). Recorded sessions never leave the machine; only aggregates are reported.

Where production's step goes. Production decodes 47 tokens/s at a 105 ms step and 4.95 accepted tokens per step. An Nsight trace splits the step into 86 ms of target GEMMs at 204 GB/s (78% of the bus), 8.6 ms of drafter, 7.5 ms of DeltaNet verify, 6.1 ms of attention over a BF16 KV cache and 3 ms of commit, KV append and sampling; the GPU idles only about 1 ms. The spec had assumed "30 tokens/s at 160 ms", which turned out to be production under two concurrent sessions on a loaded host. A side result of the same measurement: production itself gains 7% on one stream from 12 draft tokens instead of 16 (48.3 against 45.1 tokens/s), exactly, which Mightling adopted.

Decode across milestones. Table 2 follows the single-stream step. M1 reached 97% of production's decode speed with the verify's weight GEMMs already faster than production's (81.7 against 86.3 ms for the 17.6 GB) but a longer step elsewhere. M3 took 4.7 ms off with graphs (-3.0), PDL (-0.7) and an L2 prefetch in the verify attention (-1.0 to -1.5). M4's session and batch work left the one-row step unchanged within noise. The step-savings block then measured each kernel change on its own: B16, two 16-row tiles per block on the narrow matrices with two chunks in flight per warp, and the commit's DeltaNet replay moved to a second stream beside the next draft.

Prefill and request latency. v0's prefill dequantized every weight per 2,048-token chunk and ran cuBLAS, so a warm request paid 1.7 s before its first token. The block-scaled GEMM runs prefill at 2,637 tokens/s on a 1,061-token prompt, 2,584 on 8,313 and 2,106 on 34,778, against production's

Build	What changed	ms/step	tok/step	tok/s
Production (SGLang, 16 drafts)	reference	105.5	4.95	47.1
First M1 build	speculation working	146		33.7
M1	retiled weights, streaming GEMMs	114.5	5.21	45.5
M3	graphs, PDL, attention prefetch	110.5		43.5
M4 stage 4	sessions and batches; one-row path untouched	104.2	5.17	49.6
B16 (seeded)	weight stream	102.6		48.6
Commit overlap (seeded)	DeltaNet replay on a second stream	101.4	4.99	49.2

Table 2: Single-stream decode on the replayed agent sessions, server-measured. Rows before "seeded" were replayed without a fixed sampling seed and differ in acceptance between runs (section 7); the two seeded rows produced the same 17,527 tokens in the same 3,514 passes.

2,142–2,462, 1,934 and 1,683, and 3.6–4.8× M1's. On the replay a request then takes 4.30–4.43 s on average against M1's 8.14 and production's 4.18–4.25; a warm request's first token comes after 0.40 s median against production's 0.49–0.50; a session's first request after 4.4–5.2 s against 8.5–10.1, because a checkpoint after the shared system message replaces its prefill. Prompts are token-identical to production's on all 396 replayed requests tested, after three rendering differences were found and copied, one of them production's own tokenizer behaviour on combining marks.

Two streams. The M4 gate was "two agent tasks served at once at no less than production's two-stream numbers": 33.9 tokens/s per stream and a 145 ms step period. Stage 2's interleaved scheduler alone gave 22.1 tokens/s per stream (the two streams took turns); stage 3's batched verify 30.7–31.2 at a 157 ms period with 81–83% of passes batched; stage 4, which runs the drafter over both sequences' rows in one pass and every state kernel in one launch, 33.4 pooled at 151.9 ms. On seeded replays B16 took the per-stream period from 154.2 to 147.5 ms (32.6 to 34.1 tokens/s) and the commit overlap from 147.8 to 146.3 (34.0 to 34.3), both with identical output, so the engine now meets the two-stream gate. The batched step itself is about 122 ms for two sequences, against 102 for one: the second sequence costs its attention and state traffic, not a second weight read.

6 What did not pay

The context-lookup draft branch. The spec's second lever was a lookup of the continuation of the last three tokens' most recent earlier occurrence (prompt lookup [10, 13, 25]), verified in place of the drafter's chain on a long match, on the argument that 46% of agent output is copied. Scored in shadow on every step, it lost to the drafter in every match-length bucket, even on verbatim copies of 32 tokens or more (6.9 against 9.9 accepted), and the best of both per step was only +4%; the simulated ×1.2 had assumed independent successes. A later statistical study over 972 sessions (16.25M output tokens) gave the mechanism: long copies exist (45.9% of tokens in runs of 16 or more, 7.8% in runs of 128 or more) but the drafter already accepts 8–11 tokens per step on exactly those spans, so a realistic copy proposer adds 6.8% in tokens per step at 64 verify rows, and a copy step costs about as much as a drafter step. The idea survives only as a gated proposer that takes the copy when it beats the drafter, under a stricter whole-session bar.

A wider or deeper pipeline. Block 12 shortens the engine's step by 2.8 ms but loses 10% of acceptance (45.8 against 49.8 tokens/s), the opposite of production's experience; the engine keeps 16. A third `cp.async` stage in the verify attention (B07) was exact and slower at two streams (122.3 to 124.1 ms per batched step) because the kernel already read its KV at 219 GB/s against a plain read's

225. A per-sequence draft block of 16 under batching (B31) was worth about 1% and was closed. FP8 KV [11] gave no step gain on this hardware, and a wide variant of the streaming GEMM for prefill ran at a third of the speed because every 16-row weight tile re-read all 128 activation rows from L2.

What was free. Two sessions in one pass cost 17 ms over one; the drafter over both sequences' rows saved 7 ms (its weights read once); one launch per state kernel and graphs for the batch path saved nothing measurable, since at 24 rows the verify is its weight stream plus the second sequence's attention.

7 Measure the noise first

Until October 14 the replay harness sent no sampling seed, so each request sampled from a fresh random seed and two runs of "the same" replay were two workloads. The run-to-run spread in accepted tokens per step was ±5% (4.87 against 5.28 on one pair), which moves tokens/s by the same amount while moving the step period much less. That spread is the size of every gain in table 2, so B16's first two-stream pair read as a loss in tokens/s (33.9 to 33.1) while its step period had shortened by 8.8 ms with every output bit unchanged. The harness now fixes a seed per request from the run, the session and the request index; a comparison is reported only when the two runs produced the same output tokens in the same number of passes, which makes tokens/s a measurement of the step rather than of the dice. Even seeded, about one request in 144 at two streams can still differ, because resuming a session from a slot rather than re-prefilling changes the drafter's context and hence its proposals; greedy output never changes. Earlier comparisons, including production's own 47–48 tokens/s rows, carry the unseeded noise and are reported as such.

8 Status and plan

The engine serves the agent's full API (chat, the Responses conversion, reasoning split, streaming, typed tool calls, completions from ids) and the production server's metric names, so the same replay harness and the same agent run against either. Greedy top-1 agreement with production is 98.6% and top-5 agreement 100%; the mean KL against production's own run-to-run noise is within that noise. The engine has not yet replaced production: the owner's bar is a benchmark night, two agent tasks at once on real SWE-bench instances, at no less than production's numbers, and the remaining single-step savings (a context-aware sparse drafter head, 6-bit scale codes, and re-quantizing the drafter until it accepts as its BF16 original does) are measured one at a time against the main of their day. After that: structured output and images, which the agent needs; the spec's tree verify in the manner of SpecInfer [19]; and a repository-aware copy proposer under its gate. Tensor parallelism across two Sparks is last, after every one-node improvement is settled.

9 Related work

Speculative decoding descends from blockwise parallel decoding [27] and was given its exact acceptance rule by Leviathan et al. [14] and Chen et al. [2]; Xia et al. [30] survey the field. Drafters that read the target's hidden features rather than its tokens run from Medusa [1] through EAGLE and its successors [15–17] to the multi-token prediction heads trained into DeepSeek-V3 [7]; DFlash [3], the engine's drafter, proposes a whole block at once by diffusion over the same features. Tree verification [19] and dynamic draft trees [16] widen the draft; the engine verifies one chain and leaves

the tree for later. Retrieval and copy drafts, from the prompt [25], a datastore [10], or by treating copying as the model's output [13], and lookahead decoding [9] are the family the context-lookup branch of section 6 belongs to; its failure here is a statement about the drafter it competed with, not about the family. The engine's target is a hybrid of Gated DeltaNet [31, 32] and attention from the Qwen3 line [24]; commit by replay is how its recurrent state survives rejection.

That decode is bound by weight bandwidth is the roofline argument [29] applied to transformer inference [23]; the production servers the engine is measured against, vLLM [12] and SGLang [35], descend from continuous batching [33] and trade single-stream latency for throughput across many requests, which this engine does not. The weight formats are the OCP microscaling formats [22] and NVIDIA's NVFP4 [21]; the checkpoint was made with AutoRound [4], in the lineage of GPTQ [8] and AWQ [18], and the re-quantization of the drafter planned in the next section follows model-preserving rounding [28]. The attention kernels follow the IO-aware design of FlashAttention [5, 6]; the step's graph capture and programmatic dependent launch are CUDA features [20], and the persistent "megakernel" that would fuse the step further [26] is the subject of a separate study.

10 Limitations

Every number is from one machine, one model and one checkpoint, on a replay of one agent's recorded sessions; the engine is not a general server and does not try to be. Rows measured before the seed fix carry acceptance noise of the size of the reported differences, and the production rows were taken the same way. The decode measurements are server-side and warm; the agent's end-to-end gain depends on the request mix, which is dominated by short decodes. Nothing here is a leaderboard score.

Reproducibility. The engine, its specification, the milestone reports and the bench scripts are in the ling-engine repository; the recorded sessions are private and the harness skips the tests that need them.

References

[1] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.10774.

[2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[3] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.

[4] Wenhua Cheng, Weiwei Zhang, Haihao Shen, Yiyang Cai, Xin He, Kaokao Lv, and Yi Liu. Optimize weight rounding via signed gradient descent for the quantization of LLMs. *arXiv preprint arXiv:2309.05516*, 2023.

[5] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.08691.

[6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14135.

[7] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[8] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.17323.

[9] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of LLM inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.

[10] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason D. Lee, and Di He. REST: Retrieval-based speculative decoding. *arXiv preprint arXiv:2311.08252*, 2023.

[11] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2401.18079.

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[13] Tian Lan, Deng Cai, Yan Wang, Heyan Huang, and Xian-Ling Mao. Copy is all you need. *arXiv preprint arXiv:2307.06962*, 2023.

[14] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.

[15] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.15077.

[16] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2406.16858.

[17] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025.

[18] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. arXiv:2306.00978.

[19] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen,

Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating generative large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024. arXiv:2305.09781.

[20] NVIDIA. CUDA C++ programming guide: CUDA graphs and programmatic dependent launch. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`, 2025.

[21] NVIDIA. Introducing NVFP4 for efficient and accurate low-precision inference. `https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/`, 2025.

[22] Open Compute Project. OCP microscaling formats (MX) specification, version 1.0. `https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf`, 2023.

[23] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *arXiv preprint arXiv:2211.05102*, 2022.

[24] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[25] Apoorv Saxena. Prompt lookup decoding. `https://github.com/apoorvumang/prompt-lookup-decoding`, 2023.

[26] Benjamin Spector, Jordan Juravsky, Stuart Sul, Owen Dugan, Dylan Lim, Dan Fu, Simran Arora, and Christopher Ré. Look ma, no bubbles! Designing a low-latency megakernel for Llama-1B. Hazy Research blog, `https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles`, 2025.

[27] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv:1811.03115.

[28] Albert Tseng, Zhaofeng Sun, and Christopher De Sa. Model-preserving adaptive rounding. *arXiv preprint arXiv:2505.22988*, 2025.

[29] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[30] Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. *arXiv preprint arXiv:2401.07851*, 2024.

[31] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.06484.

[32] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.06464.

[33] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[34] z-lab and maurienne-ai. Qwen3.8-27B DFlash2 drafter, NVFP4 build. `https://huggingface.co/maurienne-ai/Qwen3.8-27B-DFlash2-NVFP4-RTNcal`, 2026. An NVFP4 build of `https://huggingface.co/z-lab/Qwen3.8-27B-DFlash2`.

[35] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.